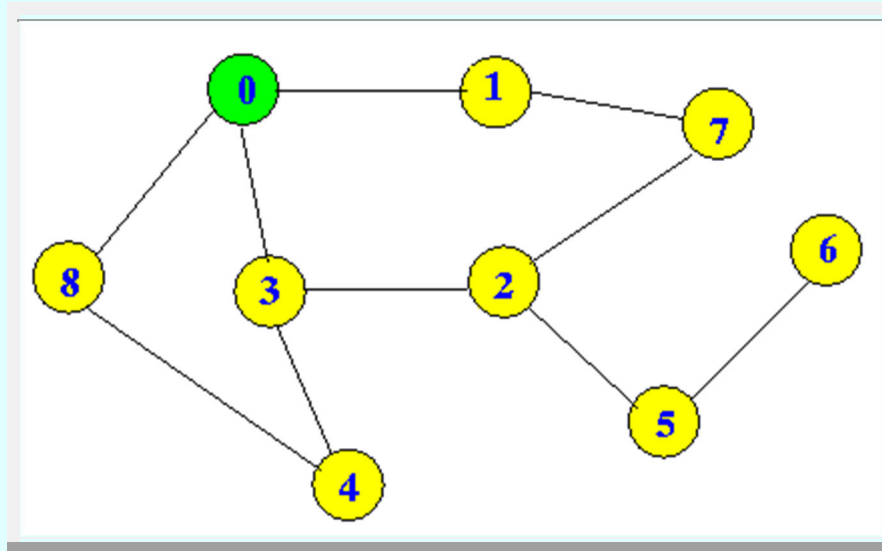


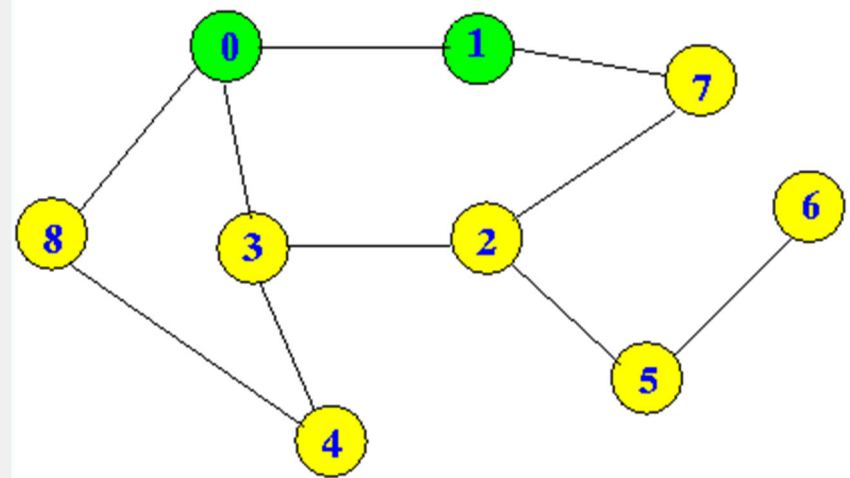
The *Depth* First Search Graph traversal algorithm

- **Depth First Search: visit nodes adjacent to the *last visited* node first**
 - **Description** of the **Depth First Search** algorithm:

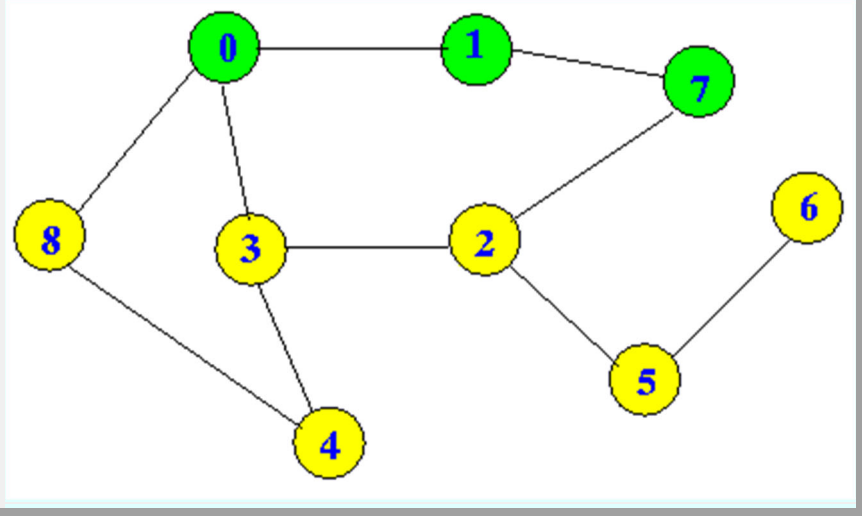
- Start at some node (e.g., **node 0**):



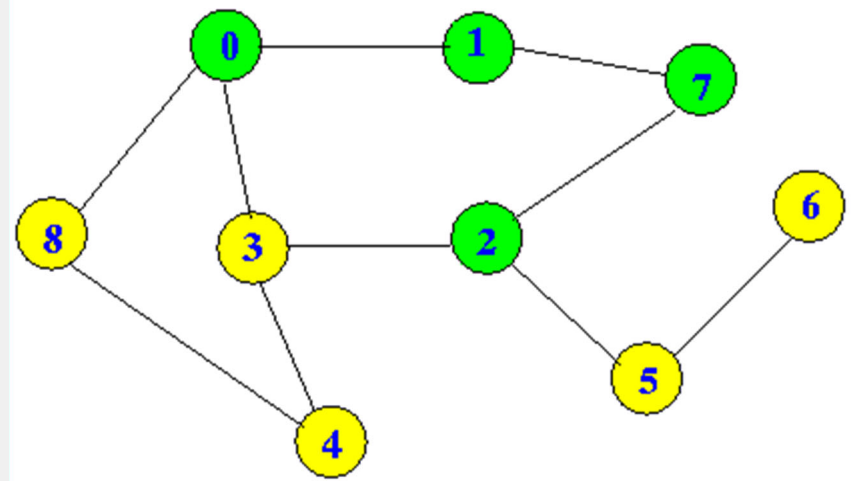
- Visit **one of the *unvisited* neighbors** of **node 0**:



-
- Then visit *one of the unvisited neighbors* of node 1:



-
- Then visit *one of the unvisited neighbors* of node 7:



-
- **And so on**

- **Implementing the DFS algorithm**
 - The **DFS** algorithm is **implemented** by:

- Using a *stack* to store the **nodes** in the `toVisitNodes` data structure.

○ **Pseudo code:**

```
Set all nodes to "not visited";

s = new Stack();    ***** Change to use a stack

s.push(initial node);    ***** Push() stores a value in a stack

while ( s ≠ empty ) do
{
    x = s.pop();    ***** Pop() remove a value from the stack

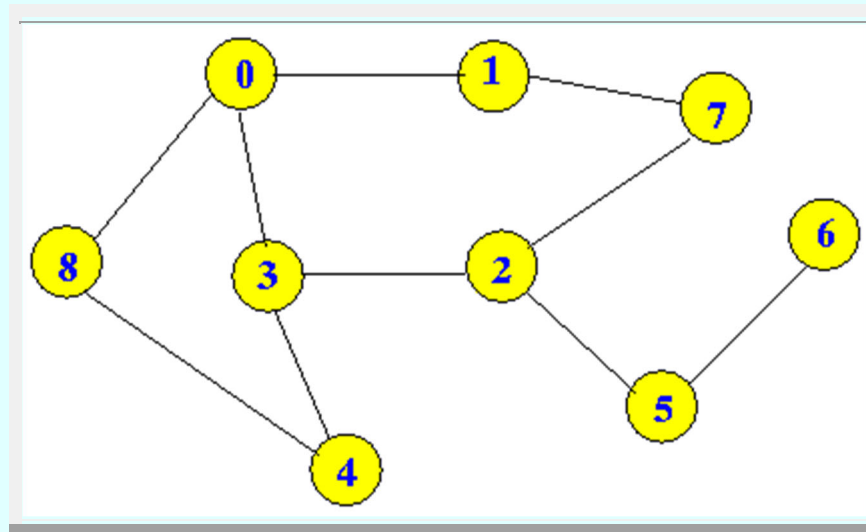
    if ( x has not been visited )
    {
        visited[x] = true;    // Visit node x !

        for ( every edge (x, y) /* we are using all edges ! */ )
            if ( y has not been visited )
                s.push(y);    ***** Use push() !
    }
}
```

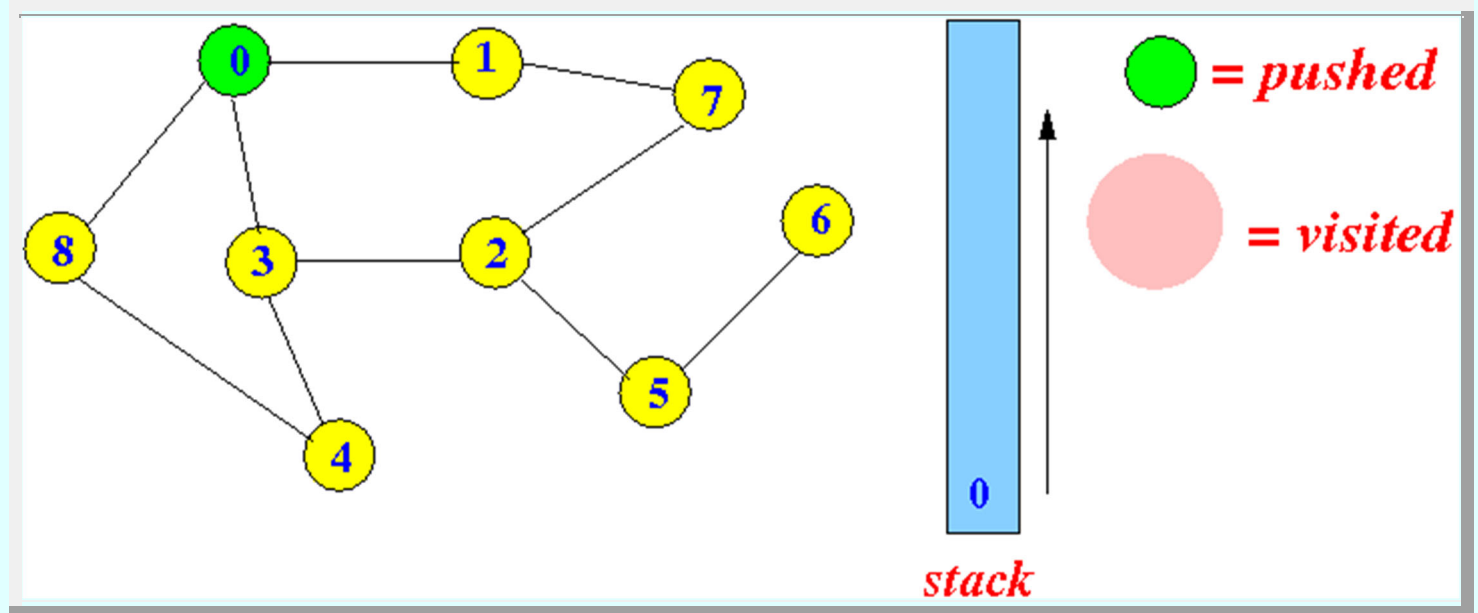
- **Example of the DFS algorithm**

- **Example:**

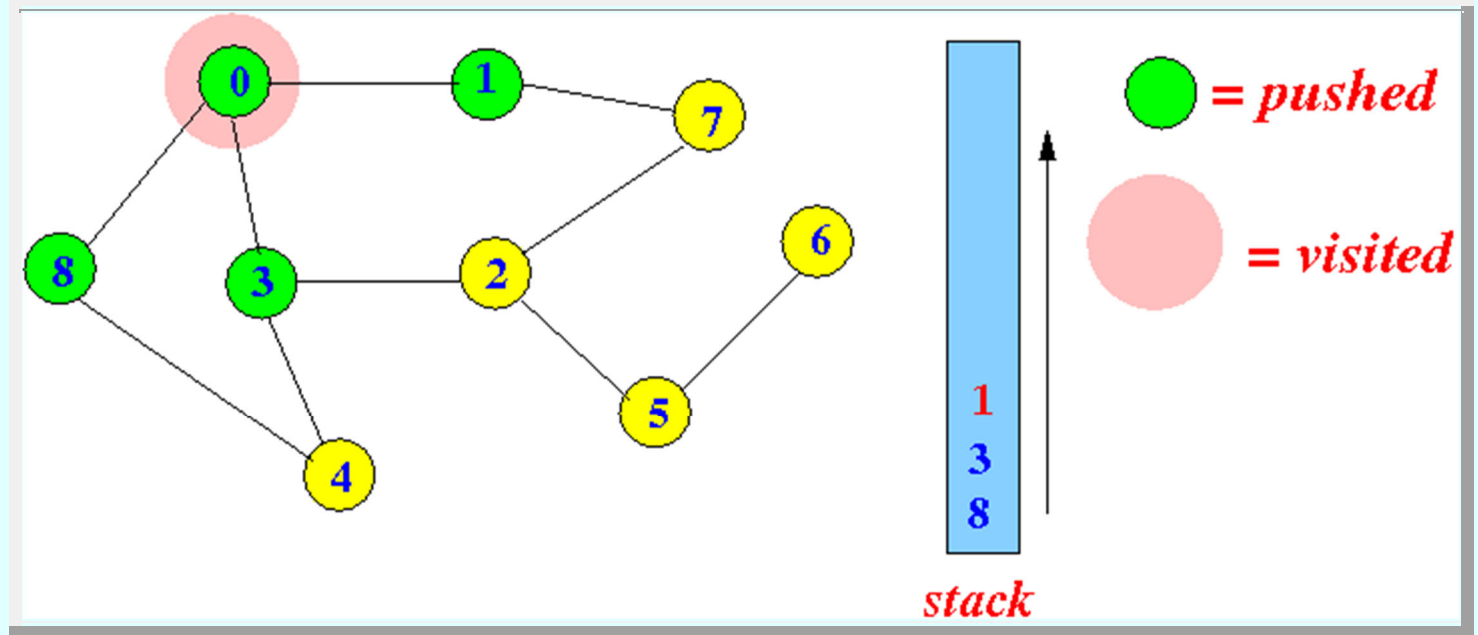
- **Graph:**



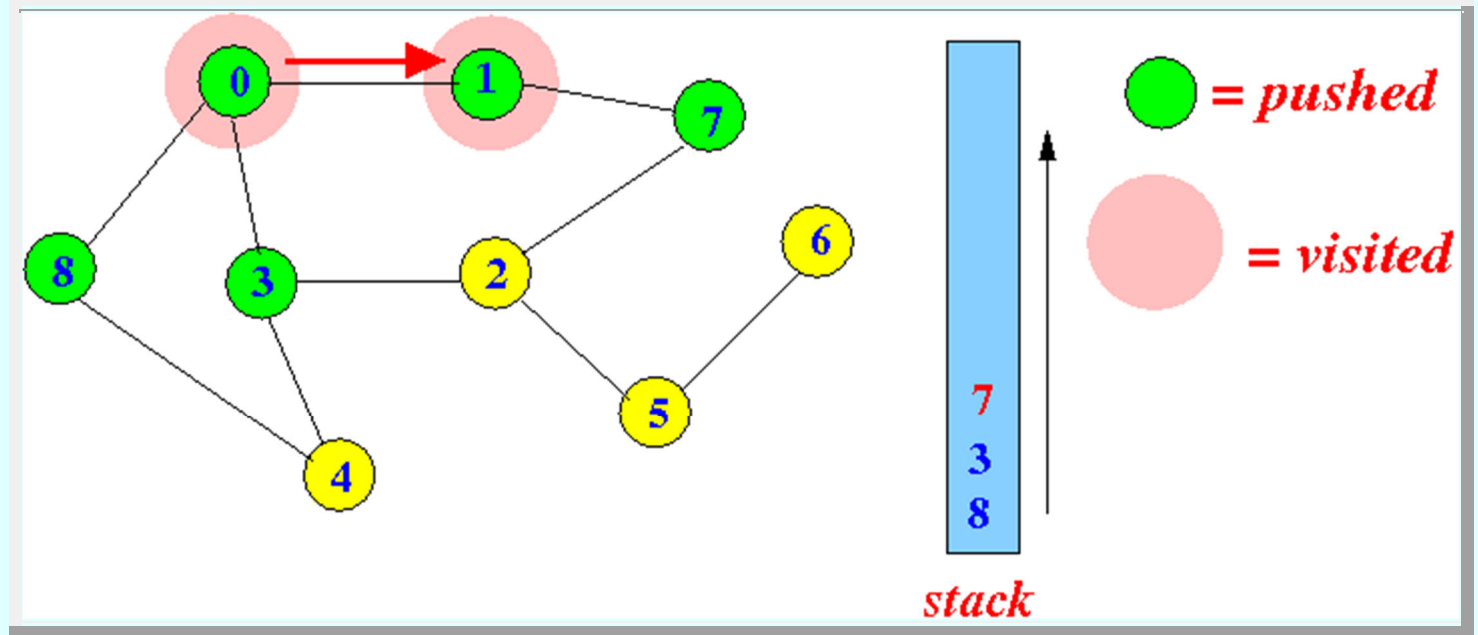
- Initial state: **node 0** is *pushed*



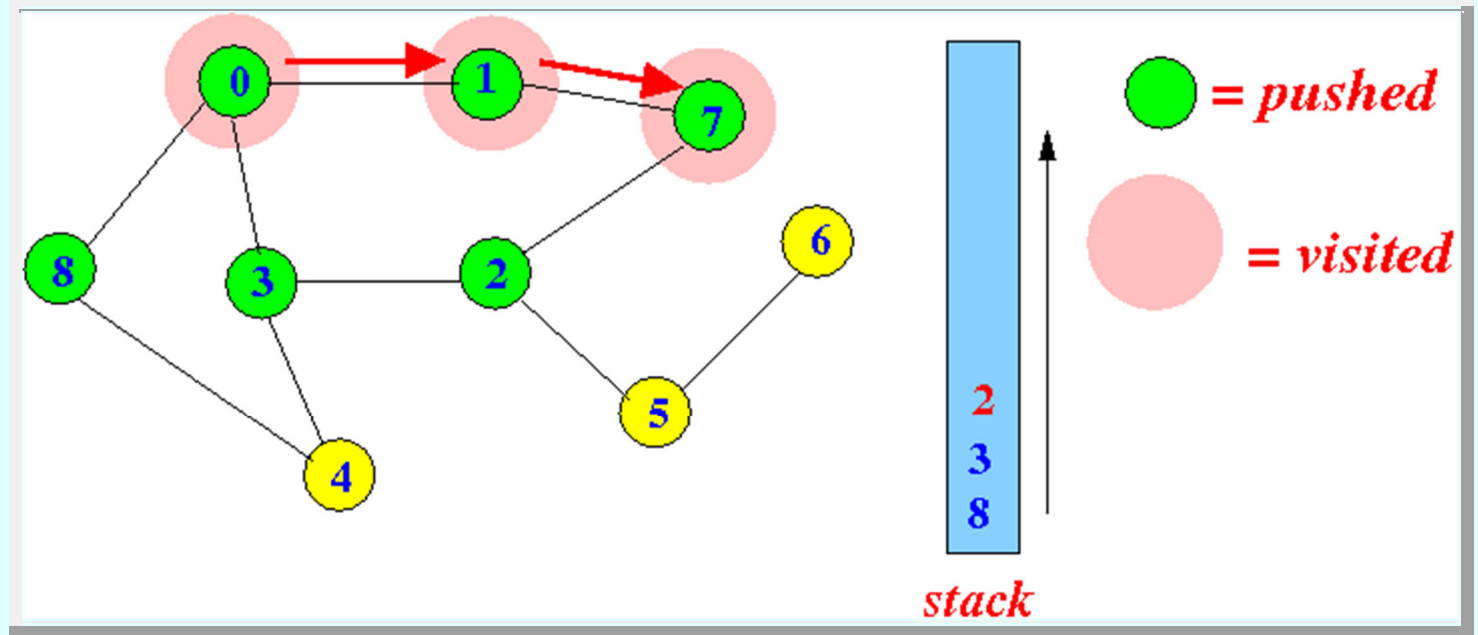
- State *after* visiting 0



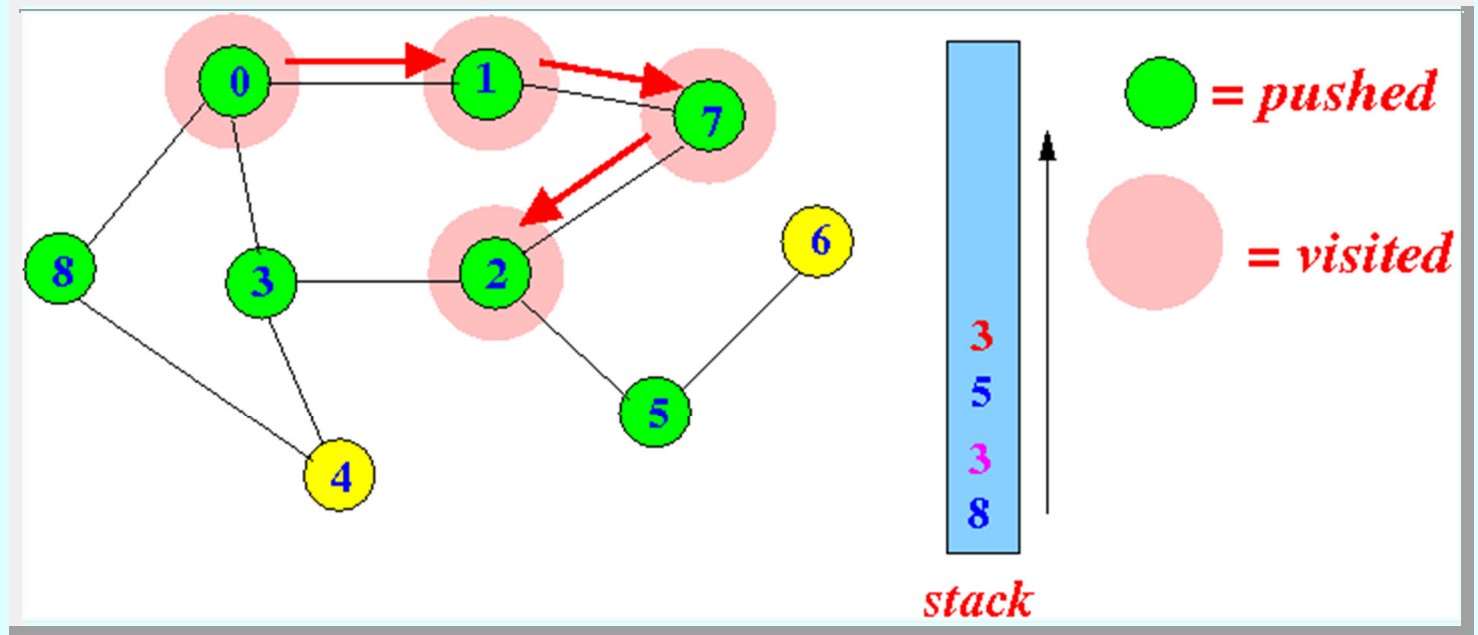
- **Push** the *unvisited neighbor nodes*: 8, 3, 1 (I used the reverse order to visit smaller node id first)
- Next, **visit** the *top node* in the **stack**: 1
- ---
- **State after visiting 1**



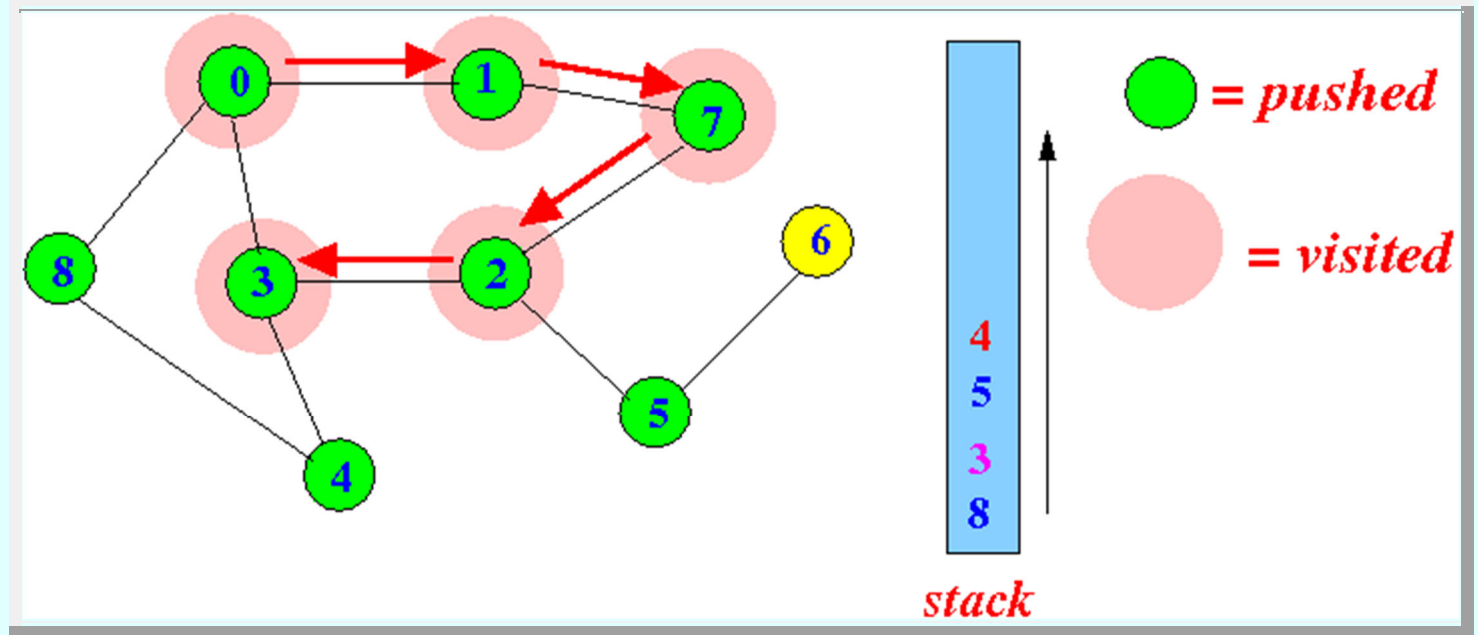
- Push the *unvisited neighbor nodes*: 7
- Next, *visit* the *top node* in the *stack*: 7
-
- State *after* visiting 7



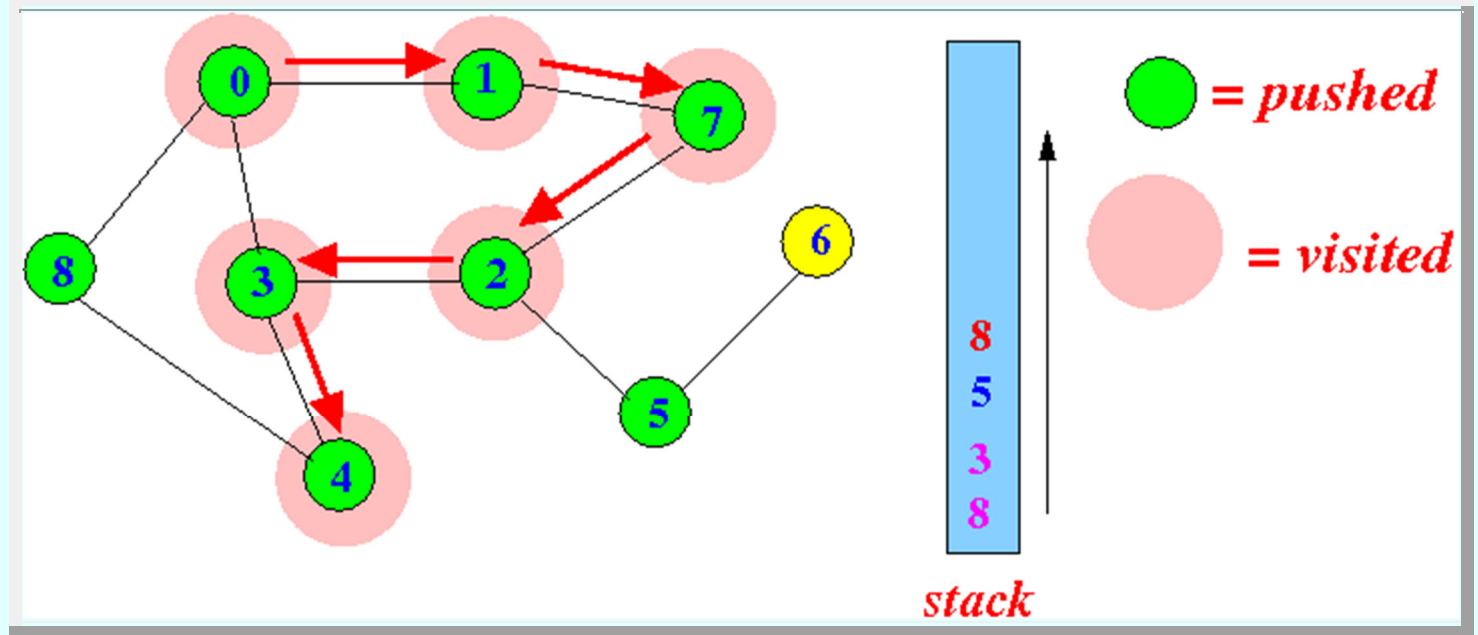
- Push the *unvisited neighbor nodes*: 2
- Next, *visit* the *top node* in the *stack*: 2
-
- State *after* visiting 2



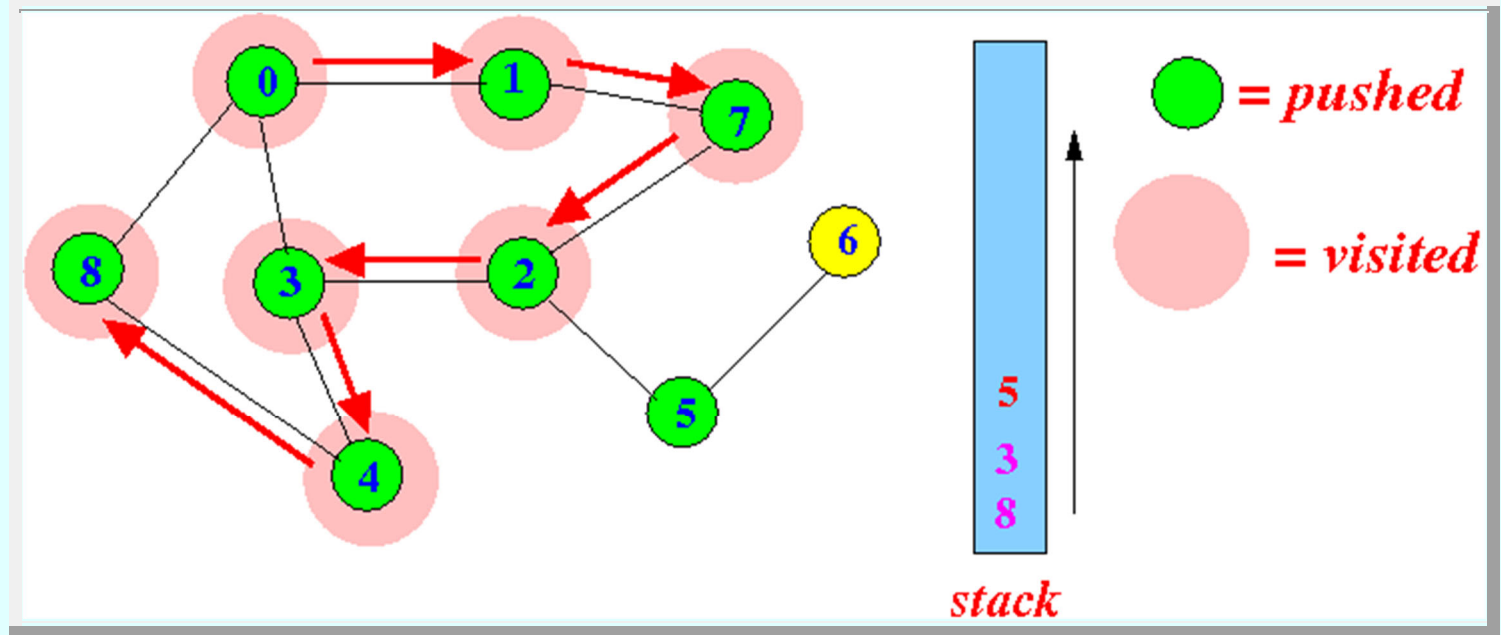
- **Push** the *unvisited neighbor nodes*: 5, 3 (Note: 3 is pushed again, and the previous value will be cancelled later -- as we will see)
- Next, **visit** the *top node* in the **stack**: 3
- ---
- State *after* visiting 3



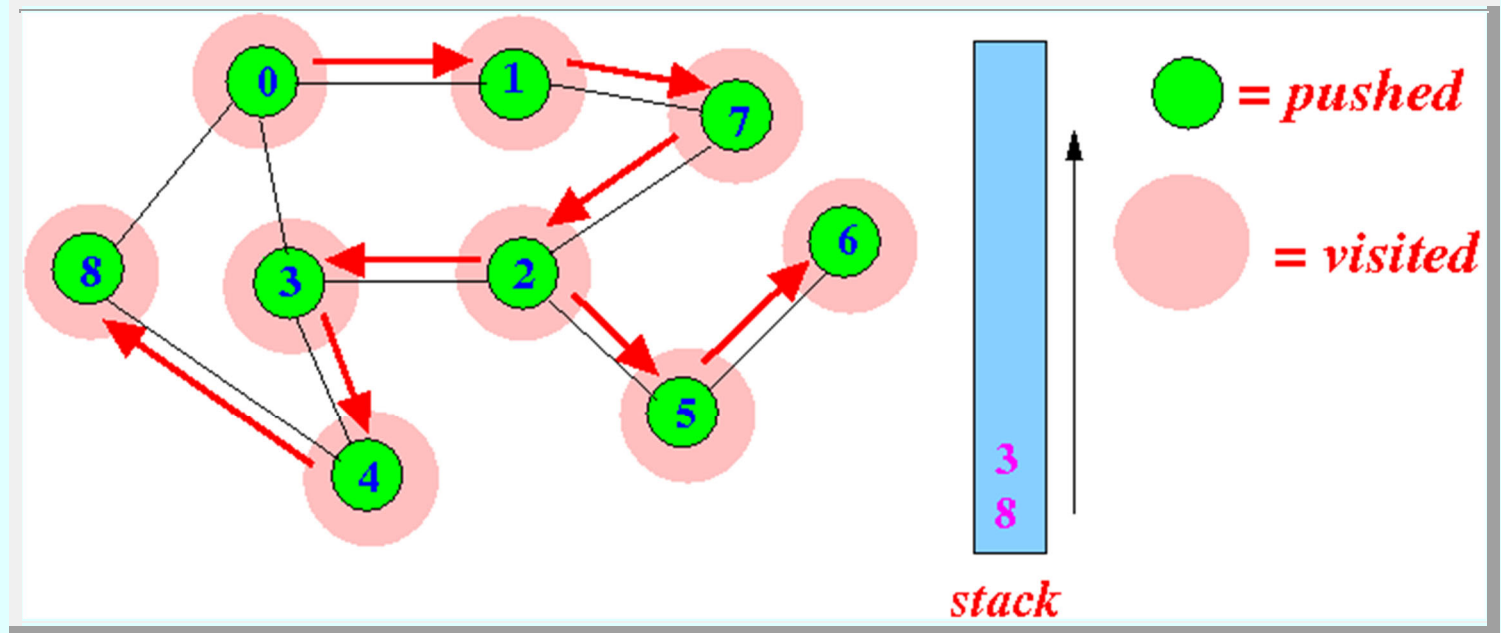
- Push the *unvisited neighbor nodes*: 4
- Next, *visit* the *top node* in the *stack*: 4
-
- State *after* visiting 4



- **Push** the *unvisited neighbor nodes*: 8 (Note: 8 is pushed again, and the previous value will be cancelled later -- as we will see)
- Next, **visit** the *top node* in the **stack**: 8
- **State after visiting 8**



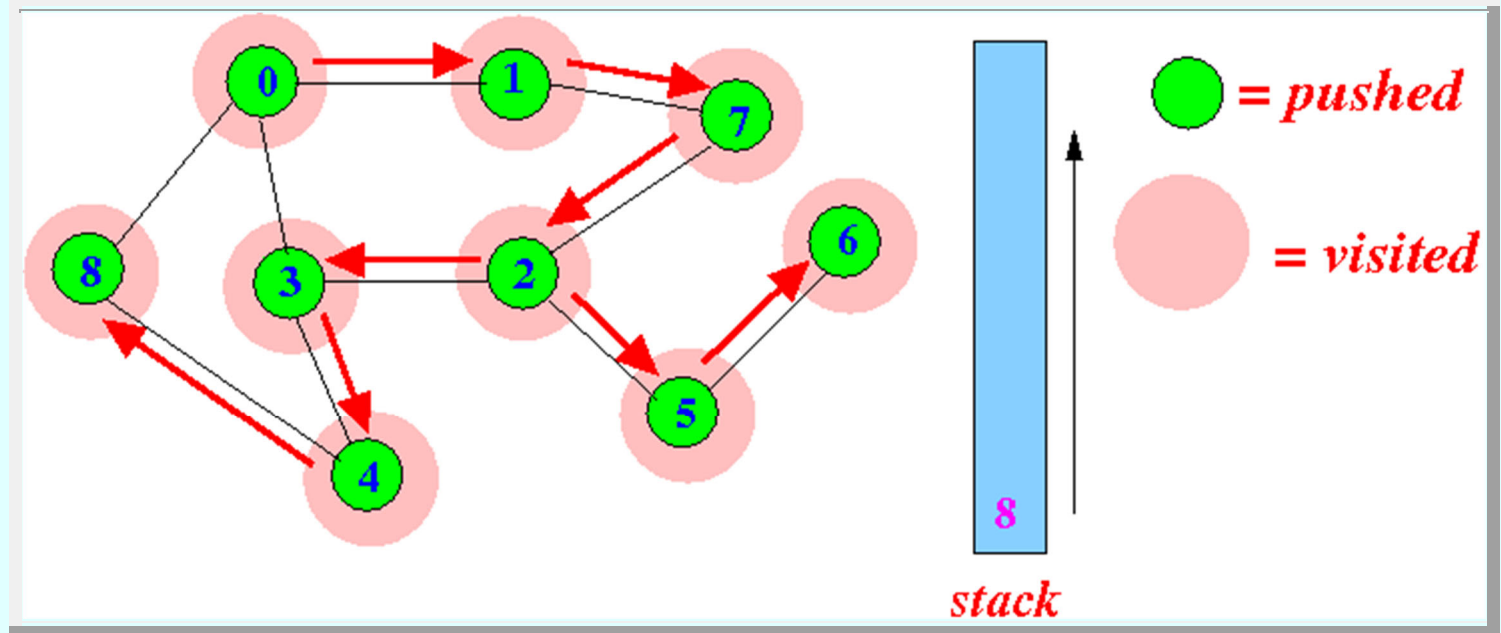
- Push the *unvisited neighbor nodes*: none
- Next, *visit* the *top node* in the stack: 5
-
- State after visiting 5



- **Push** the *unvisited neighbor nodes*: **none**
- Next, **visit** the *top node* in the stack: **3**

- **3** is *visited*: **skip**

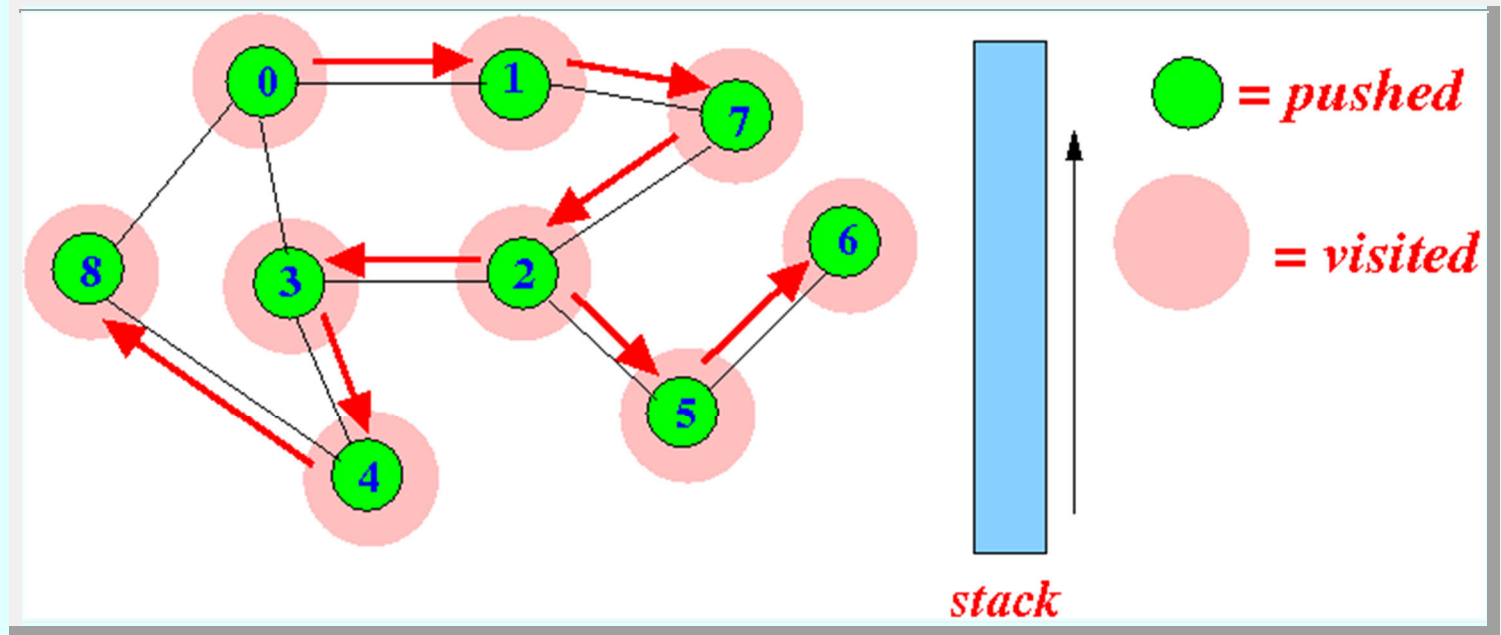
Result:



Next, **visit** the **top node** in the stack: 8

- 8 is **visited**: skip

Result:



- **DONE**

(The **stack** has become **empty**)